

Architectural Fitness Function ve Ratchet Mekanizması: Yapısal Sağlığın Sürekli Denetimi

Ömer Yavuz

May 2026

Giriş

Mimari teşhisin görselleştirilmesi ve bir kerelik düzeltilmesi, kazanılan içgörünün kalıcılığını güvence altına almaz. Sistemin geometrisi zaman içinde değişmekte, yeni özellikler eklendikçe farkında olunmayan yapısal bozulmalar birikmektedir. Bu aşamada gerekli olan, mimari ilkelerin yazılı sözleşmelerden otomatik biçimde uygulanan ölçütlere dönüştürülmesidir. Bu çalışma, söz konusu dönüşümün kavramsal çerçevesini (architectural fitness function), eşik belirleme yöntemini, geriye dönük uyumluluk mekanizmasını (ratchet) ve mevcut araç ekosistemini ele almaktadır.

Kavramsal Çerçeve: Architectural Fitness Function

Architectural fitness function kavramı, Neal Ford, Rebecca Parsons ve Patrick Kua tarafından 2017 yılında yayımlanan *Building Evolutionary Architectures* adlı eserde sistemleştirilmiştir. Tanımı şu biçimde özetlenebilir: bir mimari niteliğin (modülerlik, bağımsızlık, sınır bütünlüğü, performans, güvenlik) belirli bir hedefe ne ölçüde yaklaştığını sürekli ve otomatik olarak ölçen mekanizma. Birim testlerinin kod davranışı için oynadığı rolü, fitness function'lar mimari özellikler için oynamaktadır. İhlal anında geri bildirim sağlanır; mimari bozulmanın üretim ortamına ulaşması engellenir.

Üç Yapısal Bozulmanın Kurallaştırılması

Kod geometrisi analizinde tanımlanan üç tipik mimari koku — kardeş bağı, aşırı merkezileşmiş tip dosyası ve sınır katmanında çift yönlü baskı — doğrudan birer fitness function olarak ifade edilebilmektedir:

- Kardeş bağı: “Page niteliğindeki bir modül, başka bir page modülünü içe aktaramaz.” Bu kural, yapısal bağımlılık çözümleyicileri tarafından statik olarak doğrulanabilir.
- Aşırı merkezileşmiş tip dosyası: “Bir tip tanım dosyası, ait olduğu özellik dizini içinde kaldığı sürece belirli bir eşiğin üzerinde içe aktarma alamaz. Eşik aşıldığında dosya paylaşım çekirdeğe taşınmalıdır.”
- Sınır katmanında çift yönlü baskı: “Köprü niteliğindeki bir tip dosyası tek bir dosyada belirli bir hacmin üzerinde tutulamaz; eşik aşıldığında etki alanı bazında bölünme zorunludur.”

Bu kuralların tümü, bir bağımlılık çözümleyicinin bildirimsel yapılandırma dosyasında ifade edilebilmektedir.

Eşik Değerlerinin Belirlenmesi: Kalıp Değil, Ölçüm

Fitness function uygulamasında en sık yapılan hata, ikincil kaynaklarda geçen eşik değerlerinin (örneğin “700 satır”, “20 içe aktarma”) herhangi bir doğrulama yapılmadan benimsenmesidir. Söz konusu sayıların evrensel geçerliliği bulunmamaktadır; programlama diline, çerçeveye, etki alanı karmaşıklığına ve takım konvansiyonlarına göre belirgin biçimde değişiklik göstermektedir. Eşikler hedef kod tabanının kendi dağılımı ölçülerek belirlenmelidir.

Önerilen yöntem şu biçimde özetlenebilir:

1. Mevcut kod tabanının ilgili metriği için dağılım çıkarılır.
2. 90’ıncı yüzdebirlik (P90) değer hesaplanır; bu değer “olağan dışı ancak henüz patolojik olmayan” aralığı temsil etmektedir.
3. Eşik, P90 değerinin bir miktar üzerinde belirlenir. Bu yaklaşım mevcut yapının çoğunluğunu meşrulaştırırken yalnızca uç durumları işaretlemeyi sağlar.
4. Eşik zaman içinde aşamalı olarak indirilir; sistem sağlıklılaştıkça hassasiyet artırılır.

Ratchet Mekanizması: Geriye Dönük Uyumluluk

Bir fitness function ilk kez devreye alındığında, mevcut kod tabanında çok sayıda ihlal bulunması olağandır. Bu ihlallerin tek seferde giderilmesi pratik değildir; öte yandan kuralın yumuşatılması veya askıya alınması uzun vadede onu anlamsızlaştırmaktadır. Bu noktada uygulanan çözüm, mandal (ratchet) mekanizmasıdır:

- İlk çalıştırmada mevcut ihlal sayısı taban değer (baseline) olarak kaydedilir.

- Sonraki her pull request, ihlal sayısını yalnızca azaltabilir ya da aynı düzeyde tutabilir.
- İhlal sayısının artması durumunda derleme başarısız sayılır.
- İhlal sayısı azaldığında yeni taban değer kalıcı hale getirilir; geri dönüş engellenir.

Bu mekanizma sayesinde sistemin yapısal sağlığı yalnızca iyileşebilir; kötüleşmesi otomatik olarak engellenmiş olur. Mandalsız bir fitness function pratikte iki son-dan birine ulaşmaktadır: ya sürekli başarısızlık nedeniyle devre dışı bırakılır, ya da istisnalarla aşındırılarak işlevsizleşir.

Mevcut Araç Ekosistemi

- JavaScript ve TypeScript: `dependency-cruiser` (bağımlılık kuralları), `eslint-plugin-boundaries` (modül sınırları), `madge` (görselleştirme).
- JVM: `ArchUnit` (Java ve Kotlin için en uygun seçenek).
- Python: `import-linter` (katmanlı ve sözleşme tabanlı kurallar).
- Dilden bağımsız: özelleştirilmiş AST çözümleyici betiklerinin sürekli entegrasyon hattına eklenmesi.

Sınırlamalar: Geometri Tek Başına Yeterli Değildir

Fitness function'ların güçlü yanı, tek bir mimari boyutu (yapısal bağımlılık) sürekli ve nesnel biçimde ölçmesidir. Ancak bu güç aynı zamanda kavramın sınırlamalarının da kaynağıdır. Yapısal geometri tek başına aşağıdaki kararlar için yetersiz kalmaktadır: iş değeri ve ürün önceliği, etki alanı modelleme doğruluğu, kullanıcı deneyimi nitelikleri, çalışma zamanı performans darboğazları, güvenlik açıkları, takım kapasitesi ve teknik birikim.

Bu nedenle bir mimari karar şu bileşenlerin sentezi ile alınmalıdır: geometrik analiz, etki alanı bilgisi, test sinyalleri, çalışma zamanı sinyalleri ve ürün önceliği. Fitness function'lar bu denklemin yalnızca birinci bileşenini doğrudan ölçmektedir; geri kalan bileşenler için farklı geri bildirim mekanizmaları (etki alanı uzmanlığı, gözlemlenebilirlik altyapısı, kullanıcı geri bildirimi, planlama oturumları) gereklidir.

Sonuç

Mimari sağlığın yönetilmesi üç aşamalı bir olgunluk hattı izlemektedir. İlk aşamada sistemin geometrisi görünür kılınır; teşhis araçları ve görselleştirmeler bu aşamanın araçlarıdır. İkinci aşamada bu geometriden çıkarılan ilkeler yazılı sözleşmelere (mimari yönerge belgeleri, mimari karar kayıtları) dönüştürülür.

Üçüncü aşamada bu sözleşmeler otomatik fitness function'lar aracılığıyla sürekli denetim altına alınır ve ratchet mekanizmasıyla yalnızca iyileşme yönünde değişime izin verilir. Sürecin ulaştığı son aşamada mimari yapı tesadüfen değil; ölçülen ve aktif olarak korunan bir nitelik olarak ortaya çıkmaktadır.